

Enhancement One Narrative

CS 499 Module 3 | Software Design and Engineering | Travlr Getaways

Artifact Overview

The product chosen for enhancement one is Travlr Getaways, a travel application utilizing a MEAN (MongoDB + Express + Angular + Node.js) stack which was developed as part of the CS 465 Full Stack Development course in Fall semester of 2026. The application has a public-facing travel website that customers can use, an Express/Node.js-based API, along with MongoDB data storage using Mongoose; and a single-page Angular-based administrative application for managing trip records.

The original version of this artifact provided users with functionality to register, log-in, authenticate via JSON Web Tokens, and perform CRUD functions on trips. This artifact was selected due to its completeness as a full-stack application. It demonstrates sufficient architecture, authentication, routing, persistence, and client-server interaction to be representative of meaningful software design and engineering growth.

Rationale for Inclusion

This belongs in my ePortfolio as this shows that I can develop an entire application using all layers of the development stack. While the first version had clearly demonstrated knowledge of the various layers of the stack such as Angular Components, Express Routes, MongoDB Models, REST Endpoints, and JWT-Based Authentication; it contained the same issues typical for many young projects. In particular, it is characteristic for early stage projects that controllers are tightly bound to their respective Mongoose models, that JWT authentication has been implemented directly in the route file itself, that each endpoint handler includes redundant validation logic and that if a server fails it will potentially provide a user with internal information about the cause of the failure. Additionally, no automatic test script existed on the back end to check that necessary configurations were met when starting up the server.

I have increased the professional quality of the artifact through converting these identified problems into specific designs used in the improved version of the application. On the back end I have separated the application's routing layer from its authentication and authorization layer, validation layer, business logic layer; from its data persistence layer; and from its error handling layer. The result is a much more testable application, which is much easier to update or modify and therefore safer to expand upon in future versions.

Enhancements and Evidence

The enhancements for the software design and engineering focus on improvements in the backend architecture, how securely configured the system is, providing safe API responses, the structure of the authentication mechanism, password management, and automated testing of the artifact.

The additional artifact contains a start-up configuration module at `app_api/config/environment.js`. In addition to failing silently to a local MongoDB URI or to read `JWT_SECRET` only once when a token is generated or validated, this new feature will validate required environment variables before the application continues. If either `MONGODB_URI` or `JWT_SECRET` are missing, the application will fail quickly due to a lack of proper configuration. Thus, a better safety net has been put in place for deployments since the application will not continue to run without having defined what the JWT secret is or which database to use.

JWT authentication has been removed from an inline function that existed in `app_api/routes/index.js` and placed in reusable middleware located at `app_api/middleware/authentication.js`. This removes all of the route table logic related to defining endpoints from the responsibility of authenticating users. Additionally, since there are many routes that require authentication, it provides a common way of implementing authentication throughout the application. Trip CRUD logic has been refactored as well so that controllers do not make direct calls to Mongoose. A new flow has been implemented where controllers send requests to services and then services make calls to repositories. Validation exists in `app_api/validators/trip-validator.js` and can be reused in multiple places throughout the application. This provides clearer separation of concerns among HTTP request/response handling, business logic, input validation, and database access.

In order to provide centralized error handling, `app_api/middleware/error-handler.js` has been created along with a very small `HttpError` class. Prior to creating these enhancements, each controller contained numerous branches that would return different error messages based upon whether they were successful or not. Post-enhancements, if an expected client error occurs, a standard set of messages that include specific details about why the error occurred will be sent. However, if an unexpected server error occurs, the API will respond with a generic message stating "An unexpected server error occurred" but still log diagnostic information about the issue locally rather than exposing it in the API response. Password hashing was also modified within `app_api/models/user.js`. The previous version of the artifact used synchronous PBKDF2 with 1,000 iterations and compared hash values as strings. The updated version utilizes Node's memory-hard `scrypt` function and uses timing-safe comparisons via `crypto.timingSafeEqual`. These modifications represent a stronger approach to designing authentication mechanisms and address the previously identified security weakness in the code review. The previous version of the artifact did not contain any backend test scripts. The updated artifact includes a Node test suite utilizing `node:test`. This suite includes tests for validating configurations, validating JWT middleware functionality, sanitizing error-responses, validating payloads associated with trips, validating service functions associated with trips, checking for not-found conditions, checking for duplicate code conditions and validating fields for allowlists. Local testing passed 13 of 13 tests. The enhancement also adds a GitHub Actions quality gate at `.github/workflows/quality-gate.yml` so the same backend test suite can run automatically when the enhanced artifact is pushed or reviewed.

Decisions and Trade-offs

The major design choice was to separate the back-end into layers as opposed to creating an entirely new back-end application in addition to modifying existing database schema. An alternative way to do this would be to write an entirely new application using a different programming framework such as ASP.NET Core, Node.js, using Typescript and use a completely different method of authenticating users. However, these alternatives are outside of the scope of the software design phase and may conflict with some of the later phases including the database and algorithm development. The decision to keep the Express/Node.js and CommonJS structure of the current application while separating out the most critical parts of the application's source code via layering allows me to maintain a level of professionalism within my coding practices while keeping the overall structure of the application unchanged.

In terms of testing, I chose to focus on developing unit tests instead of spending time to develop a comprehensive end-to-end test environment using a live MongoDB database. While there are many advantages to having unit tests, they allow you to ensure the design and security aspects of your code function as expected and without relying upon the availability of a live MongoDB service. In particular, since unit tests can be executed independently from a live database service; therefore providing consistency when running said tests.

Lastly, when it comes to how I handle errors, I chose to provide sanitized error messages instead of returning raw exception details. By doing so, I improve security and privacy however, I also limit the ability for developers to diagnose issues. This is acceptable given that the primary purpose of the API is for client usage and clients should not receive internal application or system details.

Process, Challenges, and Feedback

For this milestone, the design of the backend for the enhancements of software design should be able to exist independently of the subsequent enhancements of algorithms and databases using architecture, security and testing evidence instead of simply being an opportunity to clean up some areas of the current state of the system.

An obstacle encountered during this development was determining how much of an improvement would be sufficient without creating a scope that was too large. In addition, since there will need to be additional improvements in both the algorithm and database, I deliberately chose not to change the data type of the price or duration of trips nor did I add recommendation logic in this milestone. These changes will occur in subsequent milestones. Instead, my focus was placed on establishing the backend structure that these future improvements will rely on.

A second obstacle encountered during this development was improving authentication while doing so safely and maintaining compatibility with existing Express and Passport architecture. Moving JWT verification into middleware as well as validating configuration at startup improved design without requiring replacement of the entire login workflow. Updating password hashing to scrypt and using timing safe comparisons for passwords enhanced the overall security position of the system while still allowing for better understanding in the user model. Perhaps most importantly, I learned that quality of software design is not simply related to the number of files added nor level of abstraction but rather in terms of removing specific problems within the application: repeated validation, repeated error handling, controller-to-database coupling, inline authentication, unsafe configurations and untested security behaviors.

Course Outcome Alignment

This enhancement will provide direct support for Outcome 4 and Outcome 5. Outcome 4 is being supported by using well founded software engineering methodologies, improve reliability and maintainability, which enhance the separation of concerns. Routing, authentication middleware, validation, service logic, repository access and centralize error handling have been separated into their respective areas of concern. Additionally there are automated backend tests and a GitHub actions quality gate added. The changes made here illustrate how you can apply industry best practices in software development to create a computing solution that produces an immediate value versus just cleaning up cosmetics.

Outcome 5 is being supported by utilizing security focused design enhancements. The application has been modified so that the application now validates required secrets during initialization, does not leak internal error details, utilizes reusable JWT middleware, enhances password hashing with bcrypt, provides timing safe password verification, and tests security relevant behavior. The modifications made here reflect a security first approach since they anticipate potential misconfigurations, potential vulnerabilities related to authentication and disclosure of sensitive information prior to such issues becoming defects in production.

Artifact | Travlr Getaways

Enhancement Category | Software Design and Engineering

Original Context | CS 465 Full Stack Development, 2026

Verification | 13 backend tests passing on July 19, 2026